National University of Computer and Emerging Sciences

CS2009 - Design and Analysis of Algorithms

Assignment 3

Sorting and String Searching on Real Data

Dataset: Amazon Fine Food Reviews

Name: Zoha Ashraf

Roll No: 24F-3019

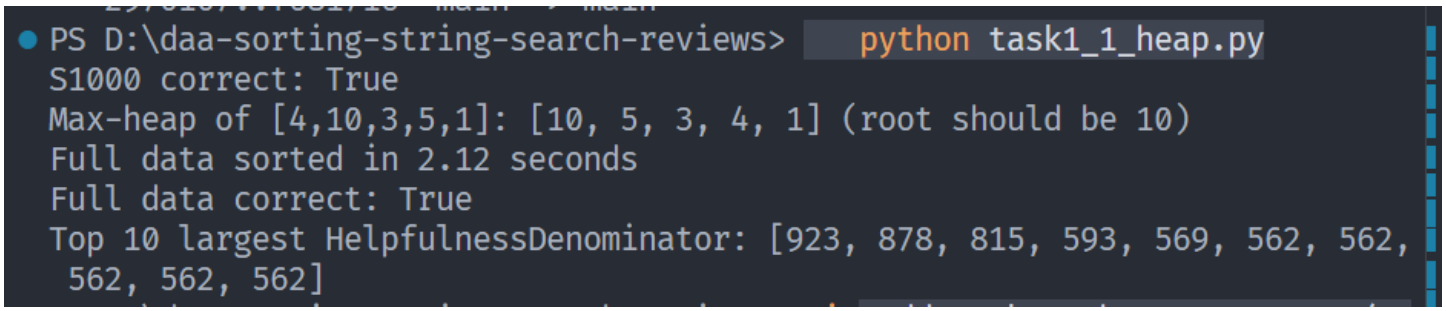
Session: Fall 2026

GitHub: github.com/ZohaAshraf/daa-sorting-string-search-reviews

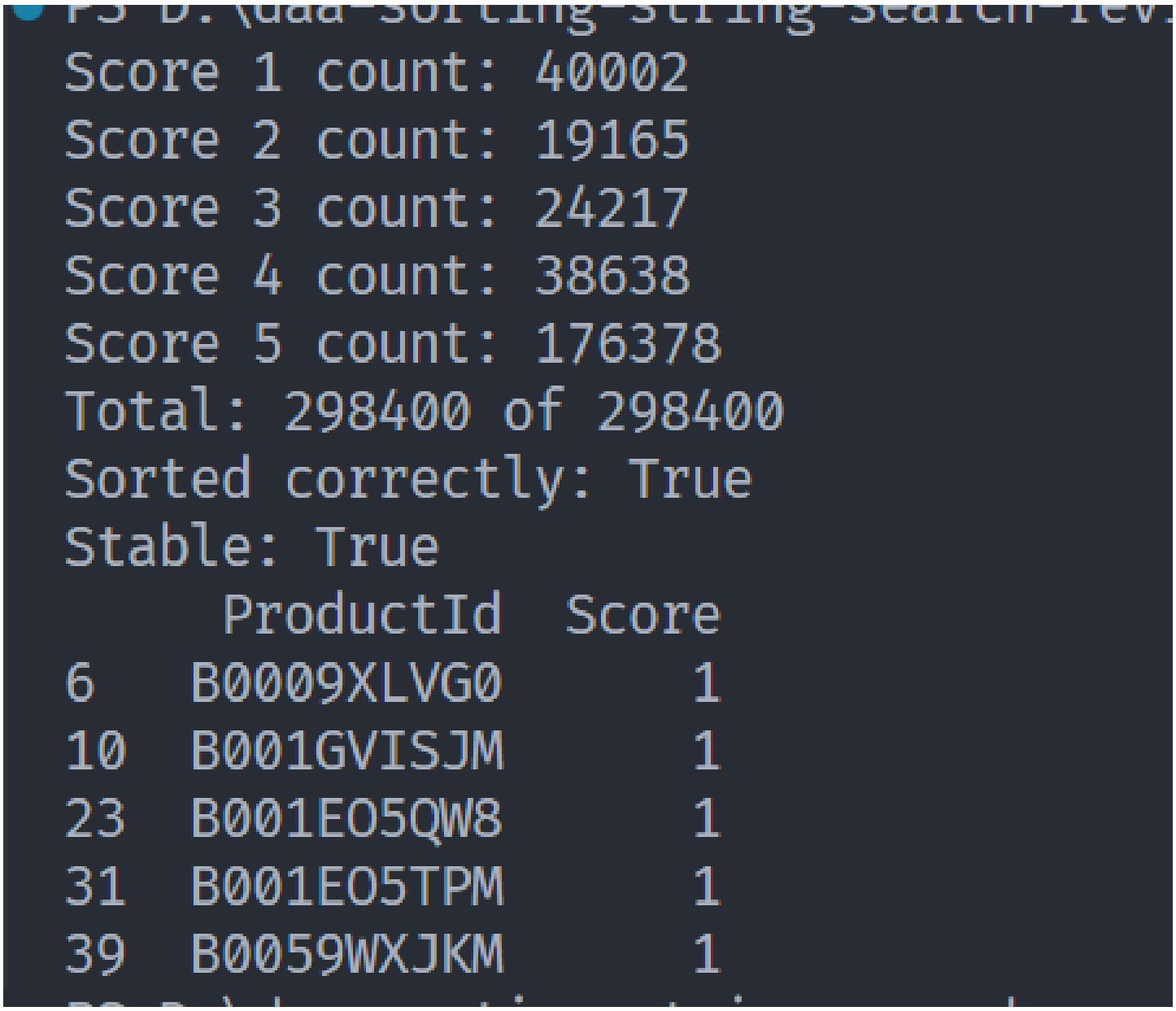
Part 0: Data Preparation output



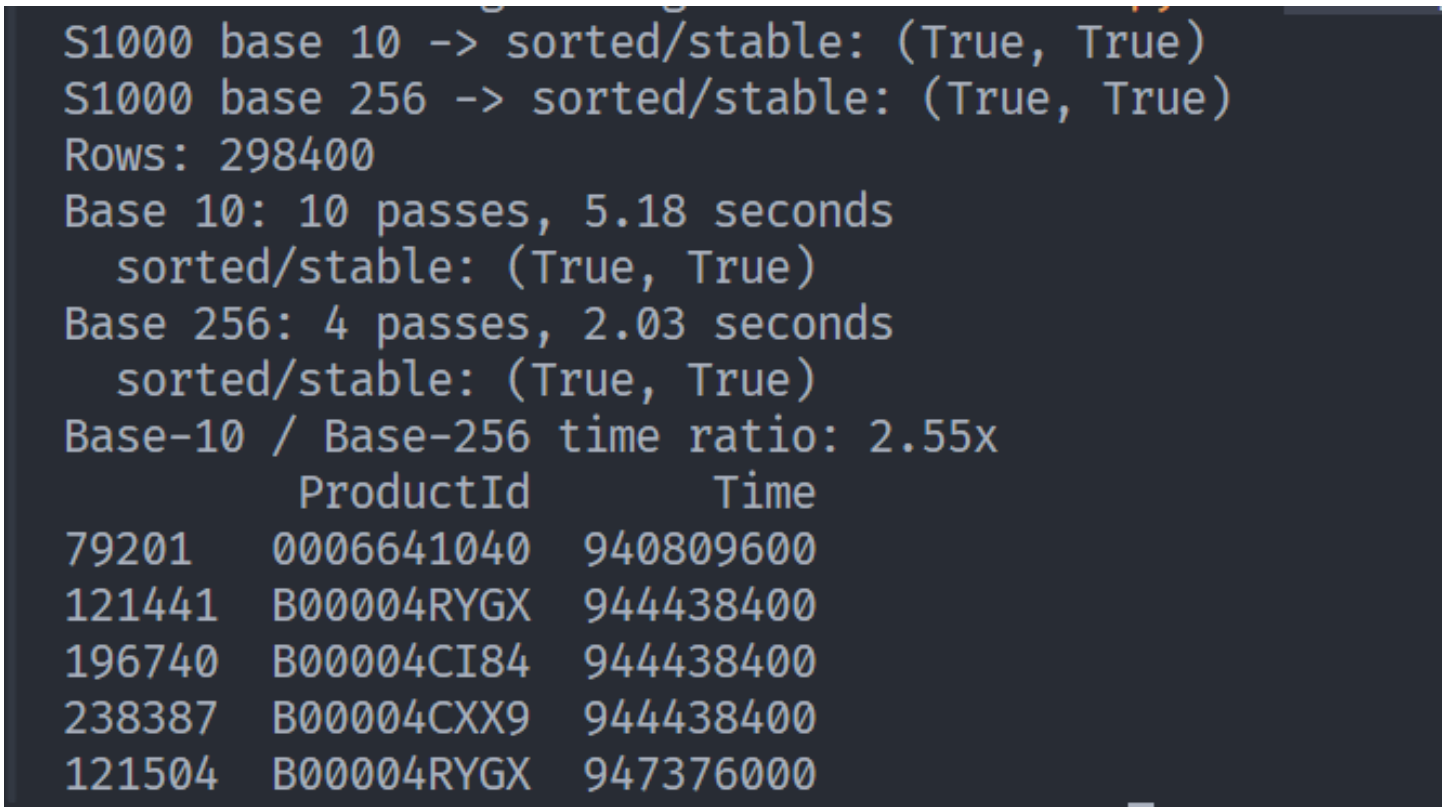
Task 1.1: Heap Sort and Top 10



Task 2.1: Counting Sort on Score



Task 2.2: Radix Sort (Base-10 vs Base-256)



```
S1000 base 10 -> sorted/stable: (True, True)
S1000 base 256 -> sorted/stable: (True, True)
Rows: 298400
Base 10: 10 passes, 5.18 seconds
  sorted/stable: (True, True)
Base 256: 4 passes, 2.03 seconds
  sorted/stable: (True, True)
Base-10 / Base-256 time ratio: 2.55x
        ProductId       Time
79201   0006641040  940809600
121441  B00004RYGX  944438400
196740  B00004CI84  944438400
238387  B00004CXX9  944438400
121504  B00004RYGX  947376000
```

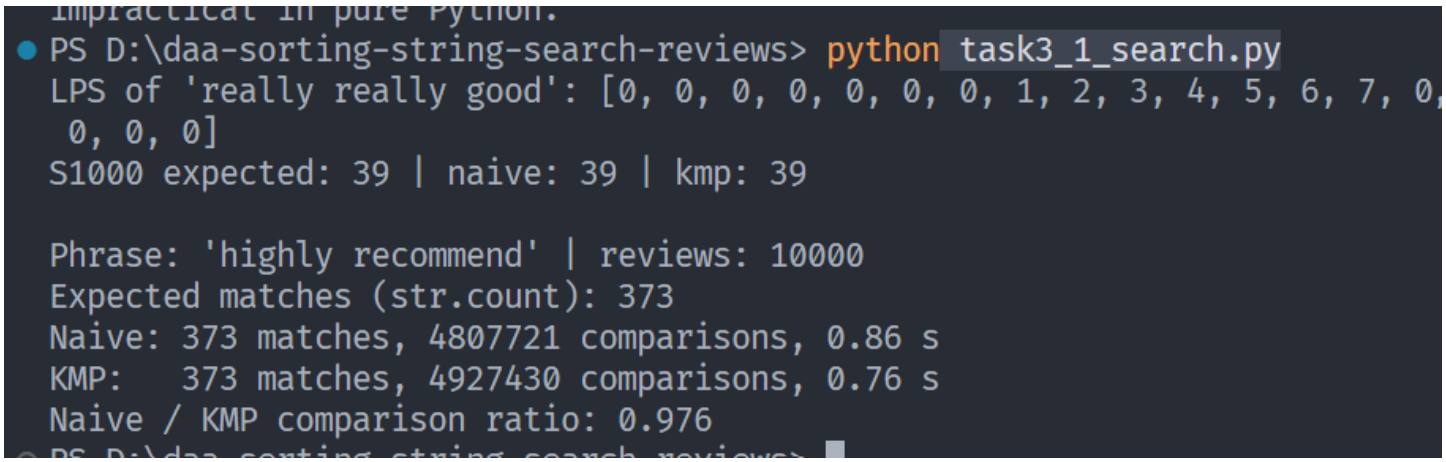
Task 2.3: Bucket Sort on ratio

```
PS D:\daa-sorting-string-search-reviews> python task2_3_bucket.py
S1000 correct: True
S1000 bucket sizes: [135, 6, 24, 24, 8, 62, 43, 33, 50, 615]

Rows: 298400
Bucket 0 [0.0, 0.1): 34518 (11.6%)
Bucket 1 [0.1, 0.2): 2248 (0.8%)
Bucket 2 [0.2, 0.3): 4353 (1.5%)
Bucket 3 [0.3, 0.4): 6246 (2.1%)
Bucket 4 [0.4, 0.5): 2748 (0.9%)
Bucket 5 [0.5, 0.6): 23166 (7.8%)
Bucket 6 [0.6, 0.7): 13916 (4.7%)
Bucket 7 [0.7, 0.8): 9548 (3.2%)
Bucket 8 [0.8, 0.9): 12112 (4.1%)
Bucket 9 [0.9, 1.0]: 189545 (63.5%)

20,000-row sample: sorted in 0.54 s, correct: True
Sample bucket sizes: [2303, 157, 335, 491, 175, 1602, 882, 615, 841, 12599
]
Note: insertion sort on the full largest bucket (189545 items) is O(n^2),
impractical in pure Python.
```

Task 3.1: Naive vs KMP



Task 3.2: Rabin-Karp

```
PS D:\daa-sorting-string-search-reviews> python task3_2_rabinkarp.py
S1000 expected: 7 | RK (q=101): (4470, 7, 4463) | RK (q=1e9+7): (7, 7, 0)

Text length: 100000
Expected matches (str.count): 0

q = 101
  Hash hits:        943
  Verified matches: 0
  Spurious hits:    943

q = 1000000007
  Hash hits:        0
  Verified matches: 0
  Spurious hits:    0

ASCII sum of 'listen': 655
ASCII sum of 'silent': 655
```

Task 4.1: Gluten Free pipeline

```
PS D:\daa-sorting-string-search-reviews> python task4_1_pipeline.py
Rows: 298400
Step 1 (KMP flagging): 27.5 s, flagged: 4083
Step 2 (radix sort): 1.6 s
ProductId sorted correctly: True
Products with >= 50 reviews: 945

Top 10 products by % of reviews containing 'gluten free':
#    ProductId    Reviews  Matches  Percent
1    B002AQP5FW   60       28       46.67%
2    B001D0676C   75       29       38.67%
3    B000EVG8H4   109      40       36.70%
4    B004391DK0   110      39       35.45%
5    B000FK63IS   53       18       33.96%
6    B001L4JH5I   93       30       32.26%
7    B000NMJWZO   195      62       31.79%
8    B0040K41MY   58       17       29.31%
9    B000K8UL4C   70       20       28.57%
10   B000DZFMEQ   120      34       28.33%

Verification: flagged matches pandas: True | valid products match: True
PS D:\daa-sorting-string-search-reviews>
```

Conceptual Answers

Task 1.1: Why heap sort is unstable

Heap sort swaps the root with the last element of the heap. That one swap carries an element past all the others in between, including ones with the same key, so equal keys can end up in a different order.

Example: [2a, 2b, 1], where 2a and 2b are equal and 2a came first. Sorting ascending with a max-heap:

- The array is already a max-heap: [2a, 2b, 1]
- Swap root with last, then fix the heap: [1, 2b, 2a] becomes [2b, 1, 2a]
- Swap root with position 1: [1, 2b, 2a]

In the result, 2b comes before 2a, so the original order of equal keys was lost.

Task 2.1 (a): Why the placement loop runs backwards

After the prefix sums, count[s] points just past the last slot for score s. Going backwards, the last item with that score takes the last slot, the one before it takes the slot before, and so on, so equal items keep their order. Going forwards, the first item takes the last slot and equal items come out reversed. The data is still sorted but the sort is no longer stable, which breaks radix sort because it needs a stable sort in every pass.

Task 2.1 (b): Why counting sort on timestamps is a bad idea

Counting sort needs one counter for every possible value. Unix timestamps in this data span about 400 million seconds, so we would need hundreds of millions of counters for only a few hundred thousand reviews. The time is O(n + k) and k is far bigger than n, so it wastes memory and is slower than a normal O(n log n) sort.

Task 2.2: Is base-256 always faster?

Not always. Base-256 needs 4 passes instead of 10, but each pass writes items into 256 different places in memory. On a large array this can cause cache misses, and with a much bigger base the count array would no longer fit in the CPU cache. So the best base depends on the machine. Base-256 is small enough to fit in cache, so it usually wins.
My times: Base-10 = 5.18 s (10 passes), Base-256 = 2.03 s (4 passes). Base-256 was about 2.55 times faster on my machine.

Task 2.3: Why one bucket is so large

Most people either vote that a review was helpful or not, and many reviews have only 1 or 2 votes. That gives ratios of exactly 1.0 or 0.0, so the values pile up at the ends, and the 1.0 values all land in bucket 9.
My bucket sizes (full data, 298,400 rows): [34518, 2248, 4353, 6246, 2748, 23166, 13916, 9548, 12112, 189545]. Bucket 9 holds 63.5% of all rows.
Bucket sort is O(n) only when items spread evenly, about n/10 per bucket. Here bucket 9 holds 189,545 items, so most of the work happens inside one bucket and the running time moves toward O(n squared). Insertion sort is cheap on runs of identical 1.0 values, but a bucket this large that also contains mixed values is too slow in pure Python. For this reason my script counted the bucket sizes on the full data and did the complete sort on a 20,000-row sample, which finished in 0.54 s and was verified correct. More buckets, or merge sort inside big buckets, would fix this.

Task 3.1 (a): LPS array for "really really good"

The pattern has 18 characters (spaces count, shown here as _).

```
index: 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17
char:  r e a l l y _ r e a l  l  y  _  g  o  o  d
LPS:   0 0 0 0 0 0 0 1 2 3 4  5  6  7  0  0  0  0
```

Working: indexes 0 to 6 are 0 because nothing has repeated yet. At index 7, "r" matches the first character of the pattern, so the value is 1. It grows by 1 each step while "really_" repeats, reaching 7 at index 13. At index 14, "g" does not match "r", so it falls back to 0, and "o", "o", "d" stay 0.
My comparison counts for "highly recommend" in 10,000 reviews: Naive = 4,807,721, KMP = 4,927,430. Both found the same 373 matches.

Task 3.1 (b): Why naive search works well on English

Naive search is slow only when long partial matches fail late, like searching for aaaaab in aaaaaaaa. In normal English a mismatch usually happens on the first or second character. For "highly recommend", most positions fail right away because the letter is not "h", so each position costs about one comparison. The O(n*m) worst case almost never appears, so KMP gains nothing here. In my run the two counts were within about 2.5% of each other, and naive was even slightly lower, because KMP's fallback steps add a few extra comparisons.

Task 3.2: Why summing ASCII values is a bad hash

A plain sum ignores the order of letters, so every anagram gets the same hash. "listen" and "silent" both sum to 655, so they look identical. Other words with the same letter total also collide. This creates many spurious hits, and each one needs a full check, so the search can slow toward O(n*m). A polynomial hash multiplies each letter by a different power of the base, so position matters and the two words get different hashes.

With q = 101 there are only 101 possible hash values, so a random window matches the pattern's hash about 1 time in 101. Over roughly 100,000 windows that predicts about 990 false hits, and I observed 943, which fits. With q = 1,000,000,007 collisions are very rare, so hash hits are almost the same as real matches.
My results (first 100,000 characters): q = 101: hash hits 943, verified matches 0, spurious hits 943. q = 1,000,000,007: hash hits 0, verified matches 0, spurious hits 0. The phrase "gluten free" does not appear in these first 100,000 characters, so every hit at q = 101 is spurious. On the S1000 test, the code found all 7 real matches, so it works correctly.

Task 4.1: Notes on the pipeline

The search matches the exact phrase "gluten free", so reviews that write "gluten-free" with a hyphen are not counted. Percentages use the cleaned dataset, so review counts per product are lower than in the raw file. KMP flags the reviews, radix sort groups them by ProductId, and one pass over the groups computes each percentage. Only the final Top 10 selection uses a built-in sort.